

Big Data and Data Engineering

Sistema de reservas de aerolínea: panel de analítica gerencial

Segundo parcial — Entrega final

Yslen Natalia Moreno Gutierrez
Oscar Mauricio Peñuela Guerrero

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 21 de octubre de 2026

Índice

1. Resumen de la solución **3**

2. Decisiones de diseño y su justificación **4**

2.1. Modelo de hosting 4

2.2. Por qué EMR 4

2.3. Organización de los datos 5

2.4. Cómo obtiene sus números el panel 6

2.5. Separación de cargas 7

2.6. Tabla de justificación de servicios 7

2.7. Proyección de costos 8

3. Ejecución del harness y resultados **9**

1. Resumen de la solución

Dato	Valor
Seed del equipo	12344
Cuenta AWS (Learner Lab)	7793-2283-8910, región us-east-1
Repositorio	**[CONFIRMAR: URL del repositorio]**
URL del sistema	`http://3.234.72.244:8000` (IP elástica)

En este parcial el sistema de reservas del primer parcial tenía que funcionar de punta a punta y además ser medible. Nuestra solución tiene dos caminos que casi no se tocan:

- **Camino transaccional:** el cliente busca vuelos y reserva desde el navegador. Una sola EC2 t3.small corre el backend en FastAPI, que también sirve las dos páginas web (`/` y `/manager`). Los datos de vuelos y reservas viven en una RDS PostgreSQL db.t4g.micro de una sola zona.
- **Camino analítico:** los 2 millones de reservas históricas llegan como CSV sucios a S3. Un job de PySpark en EMR aplica las reglas C1–C5 y escribe dos tablas en Parquet. Esas tablas están registradas en el Glue Data Catalog y se consultan con Athena.

El único punto de unión es el endpoint `GET /analytics/occupancy`, que consulta Athena y nunca la base transaccional. Así, el panel del gerente puede consultar la historia sin quitarle recursos a las reservas.

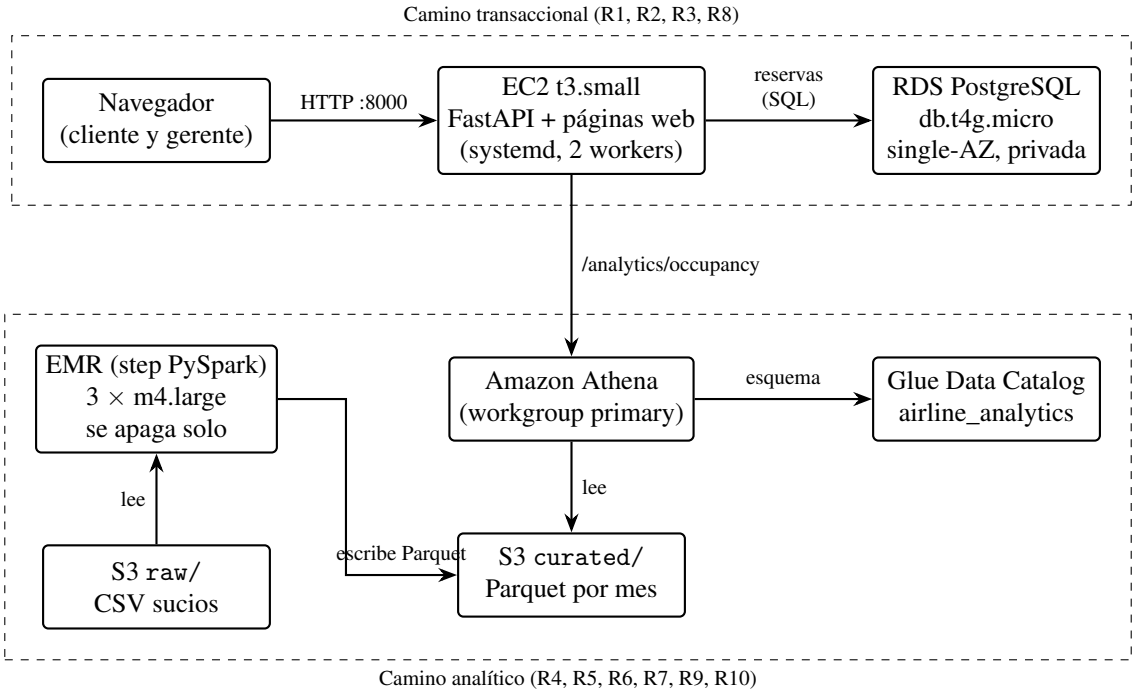


Figura 1: Arquitectura desplegada. El único cruce entre los dos caminos es el endpoint de analítica, que lee de Athena y nunca de la RDS.

2. Decisiones de diseño y su justificación

En cada decisión citamos el requisito que apoya (R1–R12) y los pilares del AWS Well-Architected Framework: excelencia operativa, seguridad, fiabilidad, eficiencia de rendimiento, optimización de costos y sostenibilidad.

2.1. Modelo de hosting

Qué escogimos. El backend y el frontend corren en **una sola EC2 t3.small** (2 vCPU, 2 GB de RAM) con Amazon Linux 2023. FastAPI corre con uvicorn como servicio de `systemd` con 2 procesos (uno por vCPU), y las páginas `/` y `/manager` son HTML y JavaScript servidos por el mismo FastAPI. La base transaccional es una **RDS PostgreSQL db.t4g.micro**, de una sola zona, sin IP pública, con 20 GB gp3 y cifrado en reposo con la llave administrada de AWS.

Por qué no algo más administrado. Consideramos varias alternativas:

- *Lambda + API Gateway:* con los 50 clientes simultáneos de R2, cada Lambda abre su propia conexión a la base. Habría que agregar RDS Proxy para no agotar las conexiones de una micro, y eso suma servicios y costo. Además el arranque en frío pone en riesgo el p95 de R1.
- *Elastic Beanstalk o ECS Fargate:* detrás suelen llevar un balanceador de carga, que cuesta unos US$16 al mes por sí solo. Eso es más que la EC2 entera.
- *Frontend en S3 + CloudFront:* suma dos servicios (R11 permite máximo 8) y obliga a configurar CORS, y no mejora ningún requisito. Servir las páginas desde el mismo FastAPI es más simple.

Por qué no algo menos administrado. Instalar PostgreSQL dentro de la misma EC2 ahorraría unos US$12 al mes, pero nos haría responsables de respaldos y parches. Además, el backend y la base competirían por los mismos 2 GB de RAM justo durante las pruebas de concurrencia de R2. RDS nos da copias automáticas (retención de 1 día, que permite restaurar la base a cualquier minuto de las últimas 24 horas), parches menores y monitoreo sin esfuerzo (excelencia operativa, fiabilidad).

La opción más barata que cumple R1 y R2. La búsqueda (R1) es una consulta por un índice compuesto `(route, flight_date)`, así que no necesita una máquina grande: el harness midió un p95 de 0,215 s (p50 de 0,099 s) con 20 clientes simultáneos, contra el límite de 1 s. Para R2 lo importante no es el tamaño de la máquina sino cómo se escribe la reserva (sección 2.5). Por eso escogimos las instancias más pequeñas permitidas en R11: optimización de costos y sostenibilidad (no pagamos ni encendemos capacidad que no se usa).

Seguridad. La RDS solo acepta conexiones en el puerto 5432 desde el grupo de seguridad de la EC2. En la EC2 solo están abiertos el puerto 8000 (la aplicación, para cualquiera) y el 22 (SSH), este último solo para el rango de EC2 Instance Connect de us-east-1 (`18.206.107.24/29`), que es como administramos el servidor desde la consola. La contraseña está en un archivo `.env` con permisos 600 dentro de la EC2 y no está en el código ni en el repositorio. La EC2 no tiene llaves de AWS: usa el `LabInstanceProfile` para hablar con Athena y S3.

2.2. Por qué EMR

Qué hace el job. El job `etl_job.py` (PySpark) lee los dos CSV crudos de la carpeta `raw/` del bucket `parcial2-aerolinea-838910`. Primero aplica las reglas en este orden:

- C1 a las rutas de los vuelos: quitar todos los espacios y pasar a mayúsculas.
- C2 a C4 a las reservas: limpiar el status, quedarse solo con CONFIRMED y CANCELLED, y quitar precios faltantes o menores o iguales a cero.
- C5: eliminar los `reservation_id` repetidos.

Después une cada reserva con su vuelo para sacar la ruta y el mes, y escribe dos tablas en Parquet: el detalle curado particionado por mes y las métricas por ruta y mes. La capacidad se suma desde la tabla de vuelos y no después del join; así cada vuelo cuenta una sola vez, incluso los que no tuvieron ventas.

Qué no podría hacer Athena sola. Athena puede consultar los CSV directamente, pero:

- Cada consulta tendría que leer y limpiar los 82 MB de texto completos. Con 10 veces más datos, eso se vuelve lento y caro en cada consulta del panel.
- Las reglas C1–C5 quedarían repetidas en cada consulta en vez de aplicarse una vez.
- "Quedarse con la primera ocurrencia" (C5) depende del orden de las filas en el archivo, y Athena no garantiza ese orden.

El job limpia una sola vez y deja datos ya listos para consultar barato.

Alternativas consideradas.

- *Athena CTAS:* podría escribir Parquet particionado, pero tiene los mismos problemas con el orden para C5 y es más difícil de probar en local.
- *AWS Glue job:* también es Spark y es serverless (sin cluster que apagar); para este volumen probablemente costaría parecido. No lo escogimos porque el curso exige EMR y porque el mismo script de PySpark lo pudimos probar en local antes de gastar horas de EMR.
- *EMR Serverless:* evita administrar el cluster, pero en el Learner Lab tiene más restricciones de permisos. Usamos EMR sobre EC2 con el tipo permitido.

Configuración y apagado (R10). Usamos un cluster de **3 instancias m4.large** (el único tipo permitido en el Learner Lab) con el job como *step* y la opción de **terminar el cluster al acabar el último paso** (confirmado en la configuración del cluster: `AutoTerminate = True`, versión `emr-6.15.0`). Así el cluster vive solo lo que dura el job y nadie tiene que acordarse de apagarlo. Antes de correrlo en EMR, validamos el script en local contra el oráculo del harness, para no gastar horas en intentos fallidos. Hicimos dos corridas (`j-0033952D23S0FE1PTYW` y `j-059278203PGLP420K06`), ambas con el step `etl` en COMPLETED y el cluster en TERMINATED, y cada una gastó 12 horas normalizadas. Esto apoya la excelencia operativa (proceso repetible), la optimización de costos y la sostenibilidad (no hay cómputo encendido sin trabajo).

2.3. Organización de los datos

Por qué Parquet. Parquet es columnar y comprimido. Una consulta que solo necesita `price_cents` y `status` lee solo esas columnas y no la fila completa. Además, los 82 MB de CSV quedan en unos 20 MB de Parquet con compresión Snappy. Como Athena cobra por byte leído, menos bytes significa menos costo y menos tiempo (optimización de costos, eficiencia de rendimiento).

Por qué particionar por mes. El panel y las preguntas de la gerencia siempre filtran por mes. Al escribir el detalle con `partitionBy("month")`, cada mes queda en su propia carpeta (`month=2023-01`, ...,

`month=2024-12`). Cuando una consulta dice `WHERE month = '2024-01'`, Athena lee solo esa carpeta (*partition pruning*). También hicimos `repartition("month")` antes de escribir, para dejar un archivo por mes en vez de muchos archivos pequeños.

Qué pasaría sin partición (R6). El harness exige que una consulta de un mes lea como máximo 2 veces el tamaño promedio de una partición:

$$\text{bytes leídos} \leq 2 \times \frac{\text{tamaño total}}{24}.$$

En nuestra corrida, la partición promedio pesa unos 819 KB, así que el límite es de 1,64 MB. La consulta de un mes del harness leyó solo 304 KB: menos de una partición, porque además de saltarse los otros 23 meses, Parquet solo lee las columnas que la consulta usa. Sin partición, Athena tendría que leer las 24 partes (unas 12 veces más que el límite) y R6 fallaría. Con CSV sería todavía peor, porque además leería todas las columnas sin comprimir.

La tabla `route_month_metrics` no está particionada a propósito. Tiene solo 240 filas (10 rutas $\times$ 24 meses), y si se particionara por mes, `month` dejaría de ser una columna normal y el contrato de R5 no se cumpliría.

Catálogo (R5). Las dos tablas se crearon con DDL en Athena (`CREATE EXTERNAL TABLE ... STORED AS PARQUET`), con los nombres y el orden de columnas exactos del anexo A.2. Las 24 particiones se registraron con `MSCK REPAIR TABLE` (paso 3 del script `setup_athena.sh`). No usamos un crawler porque puede cambiar nombres o tipos de columnas.

2.4. Cómo obtiene sus números el panel

Qué hace el endpoint. `GET /analytics/occupancy?route=&month=` manda a Athena una consulta sobre `route_month_metrics` filtrando por ruta y mes, espera el resultado y devuelve `occupancy`, `seats_sold` y `seats_capacity`, o un 404 si no hay fila. La página `/manager` solo llama a ese endpoint y muestra el porcentaje con dos decimales (por ejemplo 62,31 %). Al redondear el porcentaje a dos decimales, el error es como máximo 0,00005 en fracción, muy por debajo de la tolerancia de 0,001 de R9.

Por qué Athena y no la RDS. La historia de 2 millones de reservas no vive en la RDS, y tampoco queremos que viva ahí: cada consulta del gerente competiría con las reservas por la misma CPU (ver sección 2.5). R7 además exige explícitamente que el número salga de Athena.

Consecuencias.

- *Latencia:* en la corrida del harness cada llamada tardó entre 1,1 y 1,4 s, porque Athena tiene que planear y arrancar la consulta aunque la tabla sea pequeña. Está bien para un panel de gerencia y cumple el límite de 10 s de R7. No serviría para la búsqueda de clientes de R1.
- *Costo por consulta:* Athena cobra US$5 por TB leído, con un mínimo de 10 MB por consulta, así que cada consulta cuesta como máximo unos US$0,00005. Mil consultas al mes cuestan centavos.
- *Frescura:* los números son tan recientes como la última corrida del job de EMR (por ejemplo, una vez al mes). Las reservas de hoy no aparecen en el panel hasta la siguiente corrida. Para un panel histórico mensual esto es aceptable.

¿Se justifica una caché? Como los datos solo cambian cuando corre el job, una caché por (ruta, mes) que se borre después de cada corrida sería correcta: bajaría la latencia y el costo. Con el volumen actual el ahorro es de centavos, así que preferimos la versión más simple y consultar Athena en cada llamada. Así

también queda claro, en el historial de Athena, que el dato sale de ahí (R7). Si las consultas al panel crecieran mucho, la caché sería lo primero que agregaríamos (ver la sección de costos).

Ayudas para el usuario. Para que las páginas sean más fáciles de usar agregamos tres endpoints que no están en el contrato y que el harness no usa. `GET /flights/routes` y `GET /flights/dates` consultan la RDS para mostrarle al cliente qué rutas y qué fechas tienen cupos. `GET /analytics/catalog` le pide a Athena la lista de rutas y meses para el panel del gerente; como esa lista solo cambia cuando corre el job, se guarda en memoria una hora. El valor de ocupación, en cambio, siempre se consulta en Athena.

2.5. Separación de cargas

La regla principal es que **la analítica nunca toca la base transaccional**:

- EMR lee los CSV de S3, no la RDS.
- Athena lee Parquet de S3, no la RDS.
- El endpoint de analítica habla con Athena, no con la RDS.

Por eso, aunque el gerente haga muchas consultas o el job de EMR esté corriendo, la RDS solo atiende búsquedas y reservas (R1, R2). El trabajo pesado de la analítica lo hacen servicios separados que se pagan por uso (fiabilidad, eficiencia de rendimiento).

Cómo evitamos la sobreventa (R2). La reserva se hace en una sola transacción con un `UPDATE` atómico:

```
UPDATE flights SET seats_available = seats_available - 1
 WHERE flight_id = %s AND seats_available > 0
RETURNING flight_id;
```

- PostgreSQL bloquea la fila del vuelo mientras hace el `UPDATE`. Si 50 peticiones llegan al mismo tiempo por el último cupo, la primera lo toma, y las demás esperan y luego ven `seats_available = 0`, así que no actualizan nada.
- Si el `UPDATE` no devuelve fila, el backend responde 409 (sin cupo) o 404 (el vuelo no existe), nunca 500.
- El `INSERT` de la reserva va en la misma transacción. Si algo falla, se deshace todo y no queda un cupo descontado sin reserva.
- Como segunda protección, la tabla tiene `CHECK (seats_available >= 0)`: aunque el código tuviera un error, la base no permitiría cupos negativos.

Conexiones. Cada proceso de uvicorn tiene un *pool* de máximo 15 conexiones que se reutilizan. Si llegan más peticiones que conexiones, esperan en cola hasta 30 s en vez de fallar. Con 2 procesos son máximo 30 conexiones, por debajo del límite de una db.t4g.micro. En las pruebas del harness, las 5 rondas de 50 peticiones simultáneas (para 1 cupo y para K cupos) dieron exactamente 1 o K respuestas 201 y todas las demás 409.

2.6. Tabla de justificación de servicios

Servicio	Uso en la solución	Alternativa considerada	Requisitos	Pilares
EC2 (t3.small)	Backend FastAPI y las dos páginas web, con systemd.	Lambda + API Gateway; Elastic Beanstalk o Fargate con balanceador.	R1, R2, R3, R8, R9, R11	Costos, eficiencia de rendimiento, excelencia operativa
RDS PostgreSQL (db.t4g.micro, single-AZ)	Vuelos y reservas; UPDATE atómico contra la sobreventa.	PostgreSQL dentro de la EC2; Aurora Serverless; DynamoDB.	R1, R2, R3, R11	Fiabilidad, seguridad, costos
S3	CSV crudos, Parquet curado, script del job, logs de EMR y resultados de Athena.	Guardar la historia en la RDS.	R4, R6	Costos, sostenibilidad, fiabilidad
EMR (3 × m4.large, step + auto-terminate)	Job PySpark que aplica C1–C5 y escribe Parquet por mes.	Glue job; Athena CTAS; EMR Serverless.	R4, R10, R11	Excelencia operativa, costos, sostenibilidad
Glue Data Catalog	Esquema de `reservations_curated` y `route_month_metrics`.	Glue crawler (puede cambiar nombres).	R5	Excelencia operativa
Athena	Consultas SQL del harness y del endpoint de analítica.	Consultar la RDS; Redshift (muy caro para este volumen).	R6, R7, R9	Eficiencia de rendimiento, costos
IAM (LabInstanceProfile)	Permisos de la EC2 para Athena y S3 sin llaves en el código.	Llaves de acceso en un archivo.	R7	Seguridad

Servicios que se cobran en nuestra solución: EC2, RDS, S3, EMR, Glue y Athena. El harness cuenta además Lambda, porque la cuenta del Learner Lab trae funciones propias del laboratorio (`ModLabRole`, `MainMonitoringFunction`, `RedshiftOverwatch`, `RedshiftEventSubscription` y `RoleCreationFunction`). No hacen parte de nuestra solución y no las tocamos, porque el laboratorio las usa para administrar la cuenta. En total son 7 de los 8 permitidos en R11. No hay NAT Gateway, balanceador de carga ni Multi-AZ.

2.7. Proyección de costos

Supuestos. El backend y la base están encendidos 24/7 (730 horas al mes). El job de EMR corre una vez al mes; el cluster de 3 m4.large se apaga solo al terminar (alrededor de 0,40 USD por corrida) y dejamos margen para dos reintentos. Hay unos 2 millones de reservas (unos 110 MB en S3 sumando CSV y Parquet), pocas decenas de usuarios del panel y algunos cientos de consultas de analítica al mes. Precios on-demand de us-east-1.

Servicio	Detalle	USD/mes
RDS	db.t4g.micro (0,016/h × 730 h) + 20 GB gp3	13,98
EC2	t3.small (0,0208/h × 730 h) + EBS 8 GB gp3	15,82
EMR	1 corrida/mes de 3 × m4.large, con margen para reintentos	1,20
S3	Menos de 1 GB entre crudos, Parquet y resultados	0,05
Athena	Consultas del panel (mínimo 10 MB por consulta)	0,10
Total		**31,15**

El archivo `cost_projection.yaml` que entregamos tiene exactamente estas partidas (sin el total, porque el harness lo calcula). El total está por debajo del presupuesto de US$50. Además está cerca del costo que el harness estima a partir de lo desplegado (RDS + EC2 + horas de EMR, unos US$28), dentro de la tolerancia de 25 % de R12. El 95 % del costo son la EC2 y la RDS, que atienden las reservas; la analítica cuesta poco más de un dólar al mes porque solo se paga cuando se usa.

¿Qué pasa si los datos crecen 10 veces? Con 20 millones de reservas:

- *S3:* pasa de unos 0,1 GB a unos 1,1 GB, o sea de centavos a unos US$0,03 al mes.
- *EMR:* el job tardaría más, pero hoy la mayor parte del tiempo es el arranque del cluster, así que una corrida seguiría cabiendo en una o dos horas (entre US$0,40 y US$0,80). Si no alcanzara, se usarían más nodos m4.large por menos tiempo, a un costo parecido.
- *Athena:* la tabla de métricas sigue teniendo una fila por ruta y mes, así que las consultas del panel cuestan lo mismo. Gracias a la partición, una consulta de un mes sobre el detalle lee 1/24 de los datos y no los 10 veces más.
- *RDS y EC2:* no cambian, porque la historia no vive en la RDS.

El total quedaría en unos US$32 al mes. Esto muestra el pilar de optimización de costos (el costo crece con el uso y no con la capacidad reservada) y el de sostenibilidad (Parquet, la partición y el cluster que se apaga solo evitan leer y encender más de lo necesario).

¿Y si las consultas al panel crecen 10 veces? Athena pasaría de unos US$0,10 a alrededor de US$1 al mes, y la EC2 tendría más hilos esperando respuestas de Athena. Ahí sí se justificaría la caché por (ruta, mes) borrada en cada corrida del job: la mayoría de consultas no llegaría a Athena y el costo volvería a centavos. Si lo que crece son las reservas de clientes, el primer paso sería subir la EC2 a t3.medium (permitida en R11), unos US$15 más al mes.

3. Ejecución del harness y resultados

Cómo lo corrimos. Primero lo corrimos desde AWS CloudShell, que ya tiene las credenciales del Learner Lab, pero allí no hay navegador para las pruebas de Playwright (R8 y R9). La corrida final la hicimos desde un computador con Windows, con Playwright y Chromium instalados y las credenciales temporales del Learner Lab en `~/.aws/credentials` (que nunca se suben al repositorio). El Learner Lab cambia la IP pública de la EC2 cada vez que se reinicia, así que le asignamos una IP elástica (`3.234.72.244`). Esa es la URL que va en el `config.yaml` que entregamos, y no cambia aunque se detenga y se vuelva a encender la instancia. Lo probamos deteniendo y encendiendo la EC2: el servicio de `systemd` arrancó solo y respondió

en la misma dirección. También dejamos la EC2 y la RDS encendidas y verificamos que no hubiera clusters de EMR activos.

```
python -m pip install -r requirements.txt
playwright install chromium
python -m pytest -q # escribe report.md y report.json
```

Resultado. En la corrida final, del 22 de octubre de 2026, pasaron 22 de 23 pruebas y el puntaje automático fue de **68 de 70**. El `report.md` completo va en el repositorio.

Req.	Qué verifica	Resultado
R1	Búsqueda con p95 $\leq$ 1 s con 20 clientes	Pasa (p95 = 0,215 s)
R2	Sin sobreventa (1 cupo y K = 4 cupos, 5 rondas de 50 peticiones)	Pasa
R3	Leer una reserva (200) y una inexistente (404)	Pasa
R4	EMR COMPLETED, 24 particiones, conteo (1 997 430) y revenue exactos	Pasa
R5	Tablas en Glue con columnas exactas y particiones	Pasa
R6	Top 10 de rutas y lectura de un solo mes	Pasa (304 KB leídos)
R7	Endpoint de ocupación desde Athena en $\leq$ 10 s	Pasa (1,1–1,4 s)
R8	Flujo buscar, reservar y confirmar en el navegador	Pasa
R9	Panel igual al endpoint	Pasa
R10	Ningún EMR activo y horas $\leq$ 150	Pasa (48 horas)
R11	Instancias pequeñas, sin Multi-AZ ni NAT, $\leq$ 8 servicios	Falla parcial, 2 de 4 (ver abajo)
R12	Proyección de costos coherente con lo desplegado	Pasa

Evidencia de R2. En las 5 rondas de 50 peticiones simultáneas por el último cupo hubo exactamente una respuesta 201 y 49 respuestas 409, sin ningún 5xx. Con K = 4 cupos, las 5 rondas dieron exactamente 4 respuestas 201.

R8 y R9 (navegador). En la corrida final Playwright hizo el flujo completo en Chromium (buscar, reservar y ver el código, y el backend devolvió esa reserva). En el panel, para PSO-CLO en 2024-05, el panel, el endpoint y el valor esperado coincidieron: 0,7127.

R11 (falla parcial). La prueba de tipos de instancia encontró un `r8g.xlarge`. No pertenece a este proyecto: es el cluster `j-0520233VLLG9MZGKXFL` (*transporte-oscar*), creado el 9 de octubre de 2026, antes de empezar el parcial, para otra actividad en esta misma cuenta del Learner Lab. El harness revisa todos los clusters de EMR de los últimos 45 días y el historial de EMR no se puede borrar. Los dos clusters del parcial (`parcial2-etl`) usaron solo m4.large. Ese cluster sale de la ventana de 45 días el 23 de noviembre de 2026, y desde entonces la prueba debería pasar sin cambiar nada. Si volviéramos a empezar, usaríamos una cuenta limpia o revisaríamos el historial de EMR antes de escoger la cuenta principal. Las 24 horas normalizadas de ese cluster sí cuentan en R10, pero el total (48 de 150) sigue dentro del límite.

Nota sobre `bookings`. El oráculo del harness cuenta `bookings` como reservas confirmadas más canceladas, pero el anexo A.2 dice que `bookings` cuenta solo las CONFIRMED. Seguimos el contrato. El harness no revisa esta columna, así que no afecta el puntaje.

Partes que el harness no pudo verificar. El reporte deja una advertencia en R11: el rol del Learner Lab no tiene permiso para listar CloudFront, así que ese servicio no se contó. No usamos CloudFront (el frontend lo sirve la misma EC2), así que la cuenta real de servicios no cambia.